1. RxNorm API/ normalization of medications, autocomplete

Free
https://lhncbc.nlm.nih.gov/RxNav/APIs/index.html
Overview:
https://www.nlm.nih.gov/research/umls/rxnorm/overview.html

Integration points:
URL: [https://lhncbc.nlm.nih.gov/RxNav/APIs/api-RxNorm.getProprietaryInformation.html]
Like: getDrugs, getAllProperties, getPropNames, getProprietaryInformation
Implementing our features:
use endpoints for autocomplete, mapping to RxCUI, retrieving drug properties

How to connect:
Code snippet:

```
###
import httpx

async def autocomplete_drug(term: str, limit: int = 10):
    url = f"https://rxnav.nlm.nih.gov/REST/rxnorm.json?name={term}&max={limit}"
    resp = await httpx.get(url)
    resp.raise_for_status()
    return resp.json(
###
```

Steps to take!

1. In the FastAPI backend, building a client module (**rxnorm_client.py**) that encapsulates HTTP calls to RxNorm API.

2. Handle parameters: term search, limit, maybe filtering to prescribable content (via "Prescribable RxNorm API") for more relevant results

3. The major step is to map user inputs to RxCUI. Use the returned data to present suggestions to the user; also retrieve RxCUI and then store/use that identifier in subsequent logic (to check drug-drug interactions).

4. When storing medications in your DB you should store their RxCUI (and maybe preferred name, ingredients) to standardize across the system.

2. OpenFDA Drug API

URL: [https://open.fda.gov/apis/]
This API can give you signals of adverse events (which could include pregnancy/lactation adverse event data) and medication safety information, see more:
URL: [https://open.fda.gov/apis/drug/]

Free
URL: [[https://open.fda.gov/apis/]
You *can* use it without an API key, but if you expect significant usage (or want higher rate limits), you should request an API key. See more:
URL: [https://open.fda.gov/apis/drug/drugsfda/how-to-use-the-endpoint/]
Downloading Data: Zipped JSON
URL: [https://open.fda.gov/apis/downloads/]

Integration Points: Drug API Endpoints

adverse event data → https://api.fda.gov/drug/event.json?search=...&limit=5
URL: [https://open.fda.gov/apis/drug/event/how-to-use-the-endpoint/]

Drug approval and status → https://api.fda.gov/drug/drugsfda.json?search=products.marketing_status:"Discontinued"&limit=5

endpoint for labeling information (pregnancy and lactation warning) → drug/label

NDC directory → drug/ndc

Additional Notes:
Each endpoint supports a **search=** parameter with a specific syntax (field:term). **Limit parameter for number of records (max often 99 for some endpoints).**

The API returns JSON (or can be downloaded bulk). The JSON includes raw fields plus **openfda** harmonized fields to help.

How to use the API

Getting started with and learning how to use the API is relatively straightforward. Here are some recommended steps:

1. If you haven't already, read the API Basics documentation.
2. Review the list of searchable fields available in the Drug Adverse Event dataset.
3. Try out the Drug Adverse Event endpoint using the interactive examples and tools below.

Then, when you are ready, obtain an API Key. While you don't need an API Key to try or use the API, we recommend you get one if you are planning to use the API on a regular basis. For more information on API Keys, see the Authentication documentation.

Making a simple API Call

You can call the API from a web browser. Simply type a valid API query in your browser's address bar and press the Enter key.

In the example below, we are searching the records in the drug adverse events endpoint for matches with `headache` in the `reactionmeddrapt` field. We are requesting to see the first 5 records that match."

https://api.fda.gov/drug/event.json?search=reactionmeddrapt:"headache"&limit=5

base endpoint ? search= field:term & limit

Figure 1- How to use OpenFDA Drug API

How to connect:
In your FastAPI backend you'll create an *openfda_client.py* module. Utilize *httpx* or *requests* to call endpoints.

```
###
async def get_adverse_events(term: str, limit: int = 10):
    url = "https://api.fda.gov/drug/event.json"
    params = {"search": f"patient.drug.medicinalproduct:{term}", "limit": limit}
    resp = await httpx.get(url, params=params)
    resp.raise_for_status()
    return resp.json()
###
```

Additional Notes:
- We'll likely need to map your RxCUI or medication name to terms usable in openFDA search.

- We might choose to download bulk JSON data and locally ingest (for faster queries) or rely on live API queries.

- For pregnancy/lactation check: Using **drug/label endpoint** looking for sections pregnancy and lactation in the label to determine risk category.

Challenges:
- The openFDA data is rich but not always complete—some drugs might not have full labeling or adverse event records for the pregnant/lactation population.

- For interaction logic: openFDA is good for adverse event signals, but may not provide full drug-drug interaction mapping or pregnancy-drug interaction specifics (might come from DrugBank).

Key Points:
openFDA Drug API gives → FDA regulation / safety / adverse event / labeling data.
It complements RxNorm by providing safety and population data about drugs.
Will be integrated in the backend via HTTP calls, map drugs via RxCUI/name, and incorporate into pregnancy-interaction logic.

3. OMOP CDM- MIMIC Dataset

The OMOP CDM (Observational Medical Outcomes Partnership Common Data Model) is a standardized data model (tables + standardized vocabularies) for observational health data (EHRs, claims).
widely used in the OHDSI community.

The MIMIC (Medical Information Mart for Intensive Care) dataset is **free**.

There are efforts to convert MIMIC to OMOP format (MIMIC-IV demo in OMOP) to use OMOP tools.

URL: [https://physionet.org/content/mimic-iv-demo-omop/0.9/]

MIMIC is freely available for research.

OMOP CDM schemas are open and available (via OHDSI).

Integration Points:

In OMOP CDM, key tables relevant to our domain: person, visit_occurrence, drug_exposure, condition_occurrence, observation.

medication information is stored in **drug_exposure**.

pregnancy captured in **condition_occurrence**.

When integrating simulated or real data into OMOP, we have to map source codes to standard vocabularies → RxNorm/ SNOMED/ LOINC

How to connect:

set up a Postgres (or SQLite for ease) instance with the OMOP CDM schema (or subset of it) loaded.

Use SQLAlchemy ORM models corresponding to the OMOP tables we need Person, DrugExposure, ConditionOccurrence

In FastAPI backend we will have dependencies for DB sessions via SQLAlchemy.

```
# models.py
class DrugExposure(Base):
    __tablename__ = "drug_exposure"
    # columns: drug_exposure_id, person_id, drug_concept_id, etc.

# main.py
def get_db():
    Session = SessionLocal()
    try:
        yield Session
    finally:
        Session.close()
```

In logic: query all medications for a patient:

```
exposures = db.query(DrugExposure).filter(DrugExposure.person_id==pid).all()
```

Then map drug_concept_id to actual drug name via concept tables or to RxCUI, then feed to interaction logic.

For pregnancy or lactation → we might inspect ConditionOccurrence for pregnancy code(s) or Observation for lactation status to decide whether special logic applies.

4. DrugBank

Includes details on drug products, targets, mechanisms, interactions (drug–drug, drug–food), adverse effects.

DrugBank offers a **Drug-Drug Interaction Checker API** and a rich interaction dataset.

RxNorm's drug interaction API is powered in part by DrugBank's data.
free academic version/data (for students/research) and commercial licensing for production systems.

The API requires registration/account. Once we have credentials we can access endpoints such as **/ddi** for interactions, **/product_concepts** for autocomplete.

Integration Points for our project:
Autocomplete: **GET /product_concepts?q=tylenol** → returns product concept IDs.
DDI (Drug-Drug Interaction) search: **GET /ddi?product_concept_id=DBPC0047106,DBPC...** → returns interaction data (severity, description, management)

The API also supports searching by RxNorm codes, NDCs

The API returns JSON (structured) with fields like severity **(minor, moderate, major**), mechanism, evidence.

How to connect:
Code snippet:

In the FastAPI backend create **drugbank_client.py**.

```
###
 async def autocomplete_drug_db(term: str, limit: int = 10):
    url = f"{DRUGBANK_BASE_URL}/product_concepts"
    params = {"q": term, "limit": limit}
    headers = {"Authorization": f"Bearer {YOUR_API_TOKEN}"}
    resp = await httpx.get(url, params=params, headers=headers)
    resp.raise_for_status()
    return resp.json()

async def get_ddi(product_concept_ids: list[str]):
    url = f"{DRUGBANK_BASE_URL}/ddi"
    params = {"product_concept_id": ",".join(product_concept_ids)}
    headers = {"Authorization": f"Bearer {YOUR_API_TOKEN}"}
    resp = await httpx.get(url, params=params, headers=headers)
    resp.raise_for_status()
    return resp.json()
```

###

Use autocomplete to help user pick medications (or to map unknown input to standard product concept). Then use DDI endpoint to check interactions between meds.

For pregnancy logic: we can use the interaction severity plus any indications from DrugBank (adverse effects) and combine with openFDA/pregnancy status to generate a risk assessment.

Additional Notes:

For pregnancy/lactation, to flag drug interactions: DrugBank's DDI data is generic (drug–drug). We'll need to layer additional logic → if the patient is pregnant, severity threshold may be lower, or/ interactions escalate.

5. Fast API

FastAPI is a modern, Python-based web framework for building APIs (RESTful) with automatic generation of docs (OpenAPI), async support.

Will be used to connect to external APIs (RxNorm, openFDA, FHIR, DrugBank), handling databases (SQLAlchemy, Postgres/SQLite), exposing endpoints for front end, performing logic (check interactions and pregnancy status).
open source (MIT licensed).

To call external APIs and our own DB, use async def endpoints, integrate dependencies (database sessions, external API clients).

How to integrate:

create FastAPI endpoints like /check-interaction, /medication/autocomplete, /patient/{id}/medications.

In the endpoint logic, call external APIs (HTTP requests) or query databases (SQLAlchemy).

Use an asynchronous HTTP client to call external APIs without blocking.
set up dependency injection (FastAPI provides) for DB sessions, config for API keys, and caching layers.

6. FHIR API

FHIR (Fast Healthcare Interoperability Resources) is a standard created by HL7 International for exchanging healthcare data via RESTful APIs in modern web formats (JSON/XML).

access simulated patient data via a FHIR API, including medication information (MedicationRequest/MedicationStatement) and relevant health conditions (Condition/Observation) such as pregnancy or lactation status.

There are publicly available FHIR "sandbox" servers and simulated data sets that conform to FHIR standards.

Integration Points:

FHIR REST endpoints: **GET /Patient/{id}**, **GET /MedicationRequest?patient={id}**, **GET /MedicationStatement?patient={id}**, **GET /Condition?patient={id}&code=...**, **GET /Observation?patient={id}&code=... .**

How to connect:

In FastAPI backend build a **fhir_client.py** module:

```
###
 async def get_medication_statements(patient_id: str):

    url = f"{FHIR_BASE_URL}/MedicationStatement?patient={patient_id}"

    resp = await httpx.get(url)

    resp.raise_for_status()

    return resp.json()

###
```

7. SQLAlchemy

SQLAlchemy is a popular Python ORM (Object Relational Mapper) that allows you to define Python classes (models) mapped to database tables.
use SQLAlchemy to model database tables for: medications, drug interactions.

It's open source.

How to connect:

Code snippet:

```
###

 class User(Base):
```

```
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True)
    pregnancy_status = Column(Boolean)

class Medication(Base):
    __tablename__ = "medications"
    id = Column(Integer, primary_key=True)
    rxcui = Column(Integer, index=True)
    name = Column(String)

class DrugInteraction(Base):
    __tablename__ = "drug_interactions"
    id = Column(Integer, primary_key=True)
    rxcui1 = Column(Integer, index=True)
    rxcui2 = Column(Integer, index=True)
    severity = Column(String)
    description = Column(Text)
###
```